

LRU Cache Implementation Using Linked List

LRU stands for **Least Recently Used**.

An **LRU Cache** is a data structure that stores a limited number of items and removes the item that has **not been used for the longest time** when the cache becomes full.

Simple Example

Suppose the cache can hold **3 items**.

Initially:

Cache capacity = 3

[10] → [20] → [30]

↑ ↑
MRU LRU

Here:

- **MRU** = Most Recently Used → 10
- **LRU** = Least Recently Used → 30

If we access 20 :

[20] → [10] → [30]

↑ ↑
MRU LRU

If we now insert 40 , the cache is full. Therefore, the **LRU item 30 is removed**:

[40] → [20] → [10]

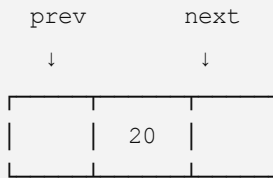
↑ ↑
MRU LRU

Why Use a Doubly Linked List?

An LRU cache is commonly implemented using a **doubly linked list** because we need to:

1. Insert an item at the front.
2. Delete an item from the end.
3. Move an existing item to the front when it is accessed.

A doubly linked list allows us to remove a node from the middle efficiently because each node has both `prev` and `next` .



The basic structure is:



Main LRU Operations

An LRU cache generally provides two operations:

1. `get(key)`

Find an item in the cache.

If found:

- Return its value.
- Move it to the **front** because it was recently used.

Example:

```
Before:
HEAD
↓
[10] → [20] → [30]
                ↑
                LRU
```

```
get(20)
```

```
After:
HEAD
↓
[20] → [10] → [30]
```

2. `put(key, value)`

Insert or update an item.

If the key already exists:

Move it to the front.

If it is a new key:

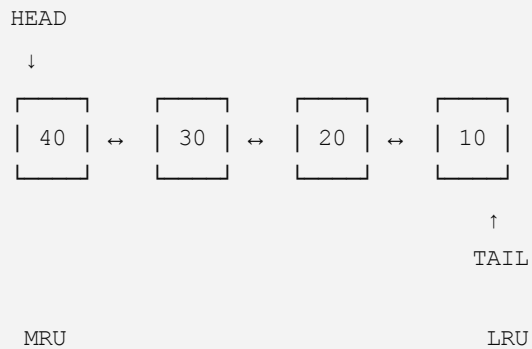
Insert at front.

If the cache is full:

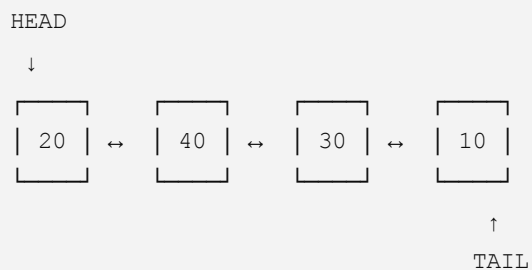
Remove the node at the tail.

Typical Structure

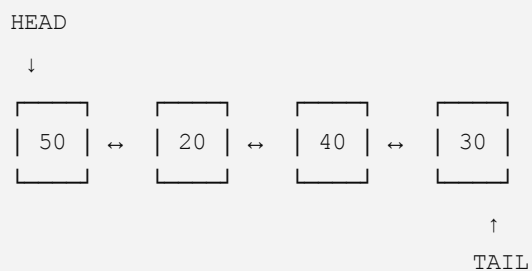
For example, with capacity 4 :



If 20 is accessed:



If 50 is inserted:



10 was the least recently used item, so it was removed.

Why Is LRU Cache Useful?

LRU caching is used when we have **limited memory** but repeatedly access some data.

Examples include:

- Web browsers
- Operating systems
- Database systems
- CPU/memory systems
- Web applications
- Image/file caching
- Frequently accessed API data

Key idea

Recently used data is kept; the least recently used data is removed when the cache is full.

Data Structures Used

For an efficient real-world LRU cache, two data structures are normally combined:

Hash Table + Doubly Linked List

The **hash table** provides fast lookup, while the **doubly linked list** maintains the usage order.

This gives approximately:

Operation	Time
<code>get()</code>	$O(1)$
<code>put()</code>	$O(1)$
Remove LRU	$O(1)$

If you are teaching **DSA**, an excellent lab exercise is to implement an **LRU Cache using a doubly linked list in C**, with a menu such as `GET`, `PUT`, `DISPLAY`, and `EXIT`.

